CHAUDHARY CHARAN SINGH UNIVERSITY (CCSU) • BCA V SEMESTER

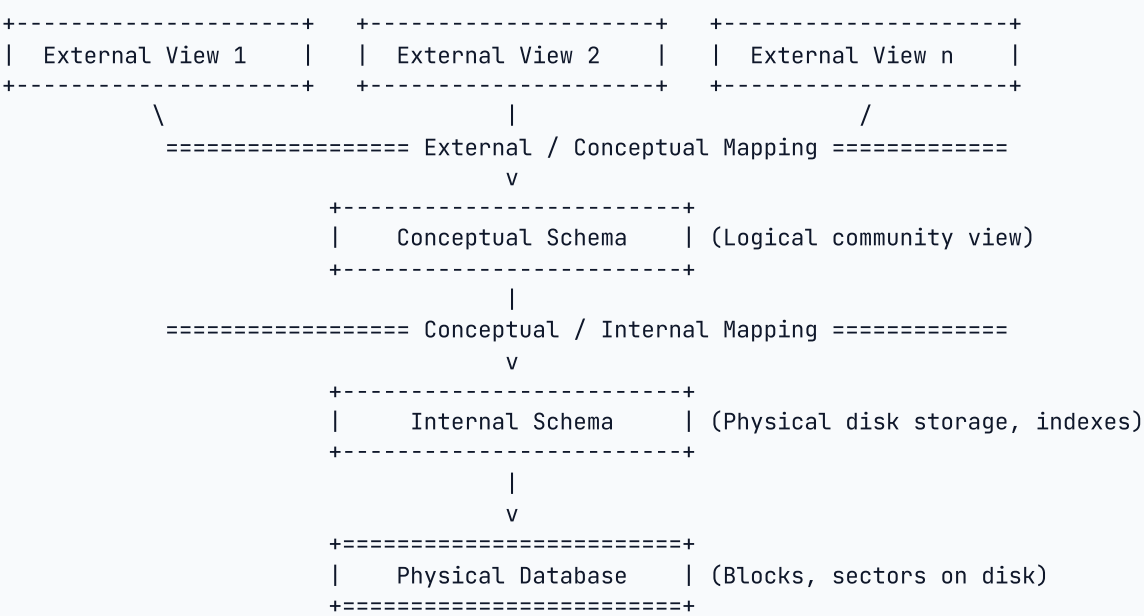
BCA-501: Introduction to DBMS

Session: December 2021 Examination | **Paper Code:** 18021 | **Authentic University Solved Paper (All Questions Solved)** | **Time:** 3 Hours • **Max Marks:** 75

SECTION A — VERY SHORT ANSWER QUESTIONS (2 × 7.5 = 15 Marks / CV-3 Pattern)

Q1 (7.5M) **Explain the Three-Level Architecture of DBMS with a neat block diagram.**

Three-Level ANSI/SPARC Architecture:



1. External Level (View Level): User-specific customized views presenting only relevant subsets of data while hiding confidential fields.
2. Conceptual Level (Logical Level): Describes *what data* is stored in the entire database and the relationships among data without physical storage details.
3. Internal Level (Physical Level): Describes *how data* is physically stored on storage media (record formats, B+ trees, hashing).

Q2 (7.5M) **Define Primary Key, NOT NULL Key, and UNIQUE Key with differences.**

Definitions & Differences:

Feature	PRIMARY KEY	UNIQUE KEY	NOT NULL
Purpose	Uniquely identifies each record.	Ensures distinct non-duplicate values.	Prevents missing or null values.
NULL Allowed	**Never** allowed (strictly 0 nulls).	Allowed (usually 1 or more nulls per column).	Does not enforce uniqueness, only disallows null.
Count per table	Only **1** per table.	Multiple unique constraints allowed.	Can be applied to every column.
Index created	Automatically creates Primary Clustered Index.	Creates Secondary Unique Non-Clustered Index.	No index created by default.

Q3 (7.5M) **Explain Hashing in brief with its collision handling techniques.**

Definition: Hashing is an efficient direct search technique where a mathematical hash function h(k) maps a search key k directly to the disk address/bucket number where the record resides, providing average O(1) time complexity.
Collision: Occurs when two distinct search keys hash to the same bucket address: h(k_1) = h(k_2).
Collision Handling Techniques:

- **Open Addressing (Linear Probing):** Searches the next contiguous sequential bucket until an empty slot is located.
- **Chaining / Separate Overflow Buckets:** Maintains a linked list of overflow buckets chained to the primary bucket.
- **Extendible Hashing (Dynamic Hashing):** Uses a directory of pointers with local and global depths to double directories and split buckets dynamically without rehashing all records.

Q4 (7.5M) **What are the pitfalls of Lock-Based Protocols in concurrency control?**

Pitfalls:

1. **Deadlock:** Occurs when two or more transactions are waiting indefinitely for locks held by each other (e.g., T_1 holds Lock on A and waits for B, while T_2 holds Lock on B and waits for A). Requires deadlock detection (Wait-For Graph) and victim rollback.
2. **Starvation:** A transaction waits indefinitely because other higher-priority transactions repeatedly acquire locks on the required data item.
3. **Cascading Rollback:** In non-strict 2PL, if a transaction aborts, all dependent uncommitted transactions that read its dirty data must also be rolled back in a cascade.
4. **Concurrency Bottleneck:** Excessive long-duration exclusive locks reduce system throughput and degrade response times.

Q5 (7.5M) **Why is Concurrency Control needed in DBMS? Explain with concurrency anomalies.**

Need for Concurrency Control: When multiple transactions execute concurrently without coordination, interference causes severe anomalies:

- **Lost Update Problem:** Two transactions overwrite each other's updates without reading the intermediate result.
- **Dirty Read (Uncommitted Dependency):** A transaction reads modified data from an uncommitted transaction that subsequently aborts.
- **Inconsistent Analysis (Non-Repeatable Read):** A transaction reads data while another transaction modifies it midway, producing skewed sums.

SECTION B — SHORT ANSWER QUESTIONS (1 × 15 = 15 Marks / CV-3 Pattern)

Q6 (15M) **Given the Relational Schema:**
EMPLOYEE(EmpID, EmpName, Age, Gender, Salary, City, DeptID, Designation, HireDate)
DEPARTMENT(DeptID, DeptName, Location)
Write SQL Queries for:
1. Display the age of 'Male' employees.
2. Display the name of the highest salary employee.
3. Display the name of department of employee 'Aman'.
4. Display all employees belonging to the same address (city).
5. Which employee is the oldest manager in the company?

SQL Queries:

```
-- Query 1: Age of Male employees
SELECT EmpName, Age
FROM EMPLOYEE
WHERE Gender = 'Male';

-- Query 2: Name of the highest salary employee
SELECT EmpName, Salary
FROM EMPLOYEE
WHERE Salary = (SELECT MAX(Salary) FROM EMPLOYEE);

-- Query 3: Department name of employee 'Aman'
SELECT D.DeptName
FROM DEPARTMENT D
JOIN EMPLOYEE E ON D.DeptID = E.DeptID
WHERE E.EmpName = 'Aman';

-- Query 4: Display all employees grouped by identical city
SELECT City, COUNT(EmpID) AS TotalEmps,
       GROUP_CONCAT(EmpName) AS EmployeeNames
FROM EMPLOYEE
GROUP BY City
HAVING COUNT(EmpID) > 1;

-- Query 5: Oldest manager in the company
SELECT EmpName, Age, Designation, HireDate
FROM EMPLOYEE
WHERE Designation = 'Manager'
ORDER BY Age DESC
LIMIT 1;
```

SECTION C — DETAILED ANSWER QUESTIONS (2 × 22.5 = 45 Marks / CV-3 Pattern)

Q7 (22.5M) **Explain Two-Phase Locking (2PL) Protocol. Compare Strict 2PL and Rigorous 2PL with serializability guarantees.**

Two-Phase Locking (2PL): A concurrency control protocol guaranteeing conflict serializability by dividing a transaction into two distinct phases:

1. **Growing Phase:** Transaction may obtain locks (Shared or Exclusive) but cannot release any lock.
2. **Shrinking Phase:** Transaction may release locks but cannot acquire any new lock.

Variants of 2PL:

- **Basic 2PL:** Guaranteed conflict serializability, but vulnerable to cascading aborts and deadlocks.
- **Strict 2PL:** All Exclusive (X) locks held by a transaction must be retained until the transaction commits or aborts. *Guarantees strict schedules and eliminates cascading aborts!*
- **Rigorous 2PL:** Both Shared (S) and Exclusive (X) locks must be retained until transaction commit/abort. Ensures serializability in commit order.

Q8 (22.5M) **Define Normalization. Explain 1NF, 2NF, 3NF, and BCNF with a complete enterprise example.**

Comprehensive Enterprise Example:
Consider unnormalized table: `STUDENT_COURSE_REG(StudentID, StudentName, CourseID, CourseName, Instructor, InstructorOffice)`.
Functional Dependencies:

- StudentID \to StudentName
- CourseID \to CourseName, Instructor
- Instructor \to InstructorOffice

Candidate Key: (StudentID, CourseID).

Step 1: Convert to 2NF (Eliminate Partial Dependencies):
- `STUDENT(StudentID, StudentName)`
- `COURSE(CourseID, CourseName, Instructor, InstructorOffice)`
- `ENROLLMENT(StudentID, CourseID)`

Step 2: Convert to 3NF (Eliminate Transitive Dependency in COURSE):
In `COURSE`, Instructor \to InstructorOffice is transitive.
- `COURSE(CourseID, CourseName, Instructor)` (FK: Instructor)
- `INSTRUCTOR(Instructor, InstructorOffice)`

All relations are now in 3NF and BCNF with zero redundancy!
